

ALEXANDRU IOAN CUZA UNIVERSITY OF IAȘI
FACULTY OF COMPUTER SCIENCE



DISSERTATION THESIS

ForestFireAI

Real-time forest fire monitoring system in forest ecosystems

proposed by:

Apricopoi Andrei-Constantin

Session: July, 2026

Scientific coordinator:

Conf. Dr. Ignat Anca

ALEXANDRU IOAN CUZA UNIVERSITY OF IAȘI
FACULTY OF COMPUTER SCIENCE

ForestFireAI

Real-time forest fire monitoring system in forest ecosystems

proposed by:

Apricopoi Andrei-Constantin

Session: July, 2026

Scientific coordinator:

Conf. Dr. Ignat Anca

Table of Contents

Motivation.....	4
Introduction.....	5
Chapter 1: Existing solutions.....	6
1.1 Human Observers	6
1.2 PanoAI	7
1.3 Dryad	8
1.4 Conclusion	9
Chapter 2: Technologies and Tools Used	10
2.1 Python and FastAPI.....	10
2.2 Ultralytics YOLOv8.....	10
2.3 OpenCV	10
2.4 Node.js and NestJS.....	11
2.5 MongoDB and Mongoose.....	11
2.6 Socket.IO	11
2.7 React and TypeScript.....	12
2.8 Redux Toolkit.....	12
2.9 JSON Web Tokens and Authentication	12
2.10 Kaggle	12
2.11 Source control: Github	13
2.12 Roboflow	13
2.13 Conclusion	13
Chapter 3: Model Training	14
3.1 Overview of the Approach.....	14
3.2 Dataset collection	15
3.3 Dataset limitations.....	16
3.4 Dataset cleaning.....	17
3.5 Labeling	18
3.6 Dataset augmentation	19
3.7 Training process.....	21
3.8 Results and evaluation	23
3.9 Conclusion	25
Chapter 4: System Architecture	26
4.1 System Overview	26
4.2 Database Structure.....	30
4.3 Python Detection Worker	31
4.4 Backend Server	32
4.5 Frontend Application.....	35
4.6 Conclusion	37

Chapter 5: Application Presentation	38
5.1 Landing Page	38
5.2 Authentication	38
5.3 Dashboard.....	40
5.4 Camera Detail Dialog.....	40
5.5 Alerts Page.....	41
5.6 Admin Cameras Page	43
5.7 Conclusion	44
Final Conclusions and Possible Improvements	44
Use of AI Tools.....	45
Bibliography	46

Motivation

Forest fires are one of the most destructive natural events on the planet, causing serious damage to ecosystems, human lives, and the global climate. In recent decades, the number and intensity of wildfires have grown considerably, driven by rising temperatures, and longer droughts.

The scale of destruction caused by forest fires is difficult to fully describe. According to data published by the European Forest Fire Information System, between 2000 and 2023, over 4 million hectares of forest were destroyed every year across Europe alone. The year 2022 was particularly severe, with more than 700,000 hectares burned across the European Union, making it the worst fire season ever recorded on the continent. [1] At a global level, NASA and the Global Fire Emissions Database estimate that between 3 and 4 billion hectares of land are affected by fire each year. [2] The 2019 to 2020 Australian bushfire season burned over 18 million hectares of land, caused the deaths of approximately 3 billion animals, and released an estimated 830 million tonnes of carbon dioxide into the atmosphere. In terms of human lives, the 2018 Camp Fire in California killed 85 people and destroyed nearly 19,000 buildings, while the 2021 fires in Greece forced tens of thousands of people to leave their homes within days.

It is safe to say that this issue could receive more mainstream attention, given the enormous damage it inflicts on the affected areas. The destruction of forests, the loss of animal and plant life, and the long term impact on air quality and the climate are consequences that affect not only the regions directly hit by fire, but the entire planet.

Given the scale of this problem, a real time monitoring system capable of detecting fire and smoke at an early stage could be of great value. Such system would be especially valuable for organizations and centers focused on monitoring vegetation and coordinating quick responses to fire incidents.

By monitoring live feeds from multiple cameras and receiving instant alerts when fire or smoke is detected, these teams could greatly shorten the time between identification and response. The timing is important for minimizing fire damage. Besides professional use, this platform could also be available to the public. It would let anyone see which areas around the world are under active surveillance and keep them updated on fire activity in areas they are concerned about.

Introduction

Technology is increasingly present in all areas of modern life. Over the last few years, the speed at which new tools and systems are being adopted across different fields has grown significantly. From healthcare and agriculture to transportation and public safety, software and artificial intelligence help solve real-world problems in ways that were not practical just a decade ago. Computer vision has made rapid progress, with object detection models achieving accuracy levels that make them suitable for practical applications.

Given the impact that forest fires have on the environment, populated areas, and the quality of life for people living near affected zones, it's natural to wonder whether modern technology could help address this issue. The consequences of a fire detected late are much worse than those caught early, and the tools needed to build an automated detection system are becoming more accessible.

With this in mind, the main goal of this paper is to create a proof of concept prototype that shows the possibility of building such tools. The project does not aim to deliver a production-ready system but seeks to explore how the different components of such a platform could work together, identify the challenges involved, and demonstrate that a functional and responsive fire detection system can be built using available technologies.

The following chapters will provide an overview of existing tools and systems in this area, explain how the proposed solution uses computer vision for fire and smoke detection, describe the process of building and training the detection model, and outline the architecture designed to support multiple users and video sources at the same time.

Chapter 1: Existing solutions

Prior to outlining the system created as part of this study, it would be beneficial to consider some of the current methods and technologies used to detect and monitor forest fires. Understanding what has been done to date will give you a better idea of how this project fits into the larger picture of fire monitoring, and also point out areas that could potentially use additional research or development.

1.1 Human Observers

For decades, forest fire detection has depended largely on human observation (see Figure 1). In many parts of the world, fire lookout towers are strategically placed on hills, mountains, or other elevated locations, giving observers a broad view of the surrounding landscape. Their task is simple in principle: watch for smoke, unusual activity, or any indication that a fire may have started. More recently, some regions have supplemented these towers with camera networks, allowing personnel to monitor large forest areas from centralized control centers.

Despite its long history, this approach faces several challenges. The ability to spot a fire often depends on conditions that are difficult to control, such as visibility, weather, and the operator's level of attention.

Monitoring multiple camera feeds for extended periods can become tiring, making it easier to overlook early warning signs. Small fires may remain undetected until they produce enough smoke to attract attention, and nighttime conditions can make detection even more difficult. Another drawback is the need for continuous human involvement. Because these systems rely on personnel being available at all times, costs can become substantial, especially across large forested regions.

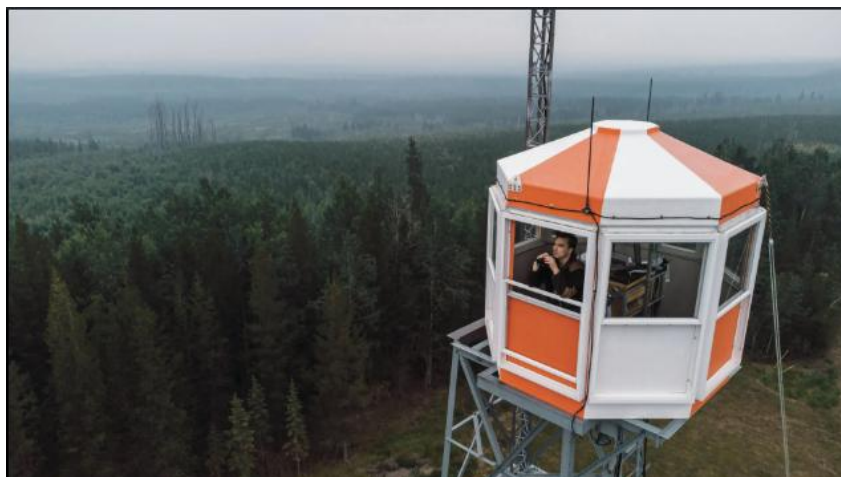


Figure 1: Human watching from tower

1.2 PanoAI

PanoAI [3] is a wildfire detection platform that uses high-quality cameras, often set up on observation towers (Figure 2), together with artificial intelligence to spot smoke as it happens. These cameras monitor wide areas, and the AI looks for signs that a wildfire could be starting. When the system finds a possible fire, it sends an alert for human operators to check before emergency services are notified.

One main advantage of PanoAI is that it can watch for fires around the clock without people having to monitor the camera feeds. This reduces the workload compared to traditional methods and helps detect fires earlier. Reports from across the United States show that the system has found wildfires faster than older reporting methods, allowing authorities to respond more quickly.

Despite these benefits, PanoAI has some limitations for research and education. It is a commercial product mainly designed for government agencies, utility companies, and other organizations with enough funding and technical support. As a result, smaller communities, independent developers, and academic researchers usually cannot access it. The details of how the detection algorithms and system work are also not shared publicly because they are proprietary.

This lack of openness makes it difficult for researchers to study the system, repeat its results, or build on its design. While PanoAI shows the potential of AI for wildfire detection, it does not provide many opportunities for open scientific research or experimentation.



Figure 2: PanoAI camera setup

1.3 Dryad

Dryad [4] takes a different approach to detection compared to camera-based systems. Instead of looking for visible signs like smoke, the company places small solar-powered sensors throughout forests (Figure 3).

These sensors are attached to trees and constantly track environmental factors such as temperature, humidity, air pressure, and certain gases that could signal a fire. By monitoring changes in these readings, the system can spot early chemical signs of a fire, sometimes even before smoke or flames appear.

The sensors use a low-power mesh network to work in remote areas. This setup lets them send information from one device to another until it reaches a central gateway, so they do not need regular internet connections. The data is then sent to a cloud platform, where it is analyzed and used to send alerts if something unusual is found.

One key benefit of Dryad's method is that it targets the very early stages of a wildfire. Finding a fire before smoke is visible gives responders more time to act, which can help limit the fire's spread and damage. This early detection solves a big problem with camera-based systems, which usually need smoke to be seen before they can spot a fire. Even with these advantages, the system has some practical challenges. To cover an area well, many sensors need to be installed, which can raise costs and require more maintenance. Also, the platform does not give visual proof of a possible fire. When an alert goes off, operators get sensor data but cannot confirm the situation right away without using other tools like cameras or checking the site in person. Because of this, Dryad works best when used with other monitoring systems, not on its own.



Figure 3: Dryad sensor attached to a tree

1.4 Conclusion

The solutions discussed earlier each take a different approach to fire detection and come with their own trade-offs. Human monitoring is flexible but limited by attention span and staffing costs. PanoAI offers automated visual detection on a large scale, but it is a proprietary product. Dryad allows for very early detection using environmental sensors, though it does not provide visual feedback and needs physical hardware to be installed.

The system developed in this project is not meant to compete directly with current solutions. Instead, this work looks at how to build a working monitoring platform that combines automated video analysis with a real-time web interface, using tools and technologies that are easy to access. The main goal is to explain what is needed for such a system and to point out the challenges faced during development.

Chapter 2: Technologies and Tools Used

Every tool used in this project was chosen for a reason. This chapter goes through each of them, explains what role it played, and gives some context for why it made more sense than the alternatives.

2.1 Python and FastAPI

The detection worker was built using Python [5]. Python has a well-developed ecosystem for machine learning, and most key libraries use Python as their main interface. Using another language would have made things much harder because of fewer library options. We used FastAPI [6] to create an HTTP control interface, which lets the backend start and stop camera threads as needed. FastAPI is quick to set up, checks requests automatically, and needs little configuration. These qualities make it a good fit for services with only a few admin calls.

2.2 Ultralytics YOLOv8

This project's object detection model is based on Ultralytics' YOLOv8 [7]. You Only Look Once, or YOLO [8], is a family of real-time object detection models that are renowned for their speed and precision. One of the most popular variants in this family, YOLOv8, provides a simple interface for training models on fresh datasets. Its performance, vibrant community, and ease of integration into a Python program are the reasons it was selected. The model was utilized directly inside the detection worker to examine video frames after being trained on a custom dataset that included pictures of smoke and fire.

2.3 OpenCV

The detection worker uses the open-source computer vision package OpenCV [9] to process video files. It was used to read individual frames at regular intervals, open video files, and store annotated pictures on disk following detection. One of the most popular tools for this type of work is OpenCV, which easily interacts with NumPy and Python.

2.4 Node.js and NestJS

NestJS [10], a framework for developing server-side applications with Node.js [11] and TypeScript, was utilized in the construction of the platform's backend. NestJS is built on a modular architecture that is modeled after Angular. This architecture makes it simple to divide the codebase into distinct modules for authentication, camera management, detections, alarms, and WebSocket communication functions. In comparison to more straightforward alternatives, it was selected because the structure that it imposes becomes more beneficial as the complexity of the program increases.

2.5 MongoDB and Mongoose

The database that was utilized for the project was designated as MongoDB [12]. It is a document-based database that stores data in a flexible format similar to JSON. This format was suitable for the nature of the data that was involved, particularly for storing camera setups and detection results that did not adhere to a conventional relational structure. Mongoose [13] was utilized as the interface between NestJS and MongoDB. It was responsible for providing schema definitions and data validation for the documents that were saved. As a result of the database being hosted remotely through the use of MongoDB Atlas [14], the requirement to manage a local database installation was no longer needed.

2.6 Socket.IO

In order to facilitate communication in real time between the frontend and the backend, the Socket protocol was utilized. It is [15]. Once a new detection is processed or a new alert is produced, the backend sends an event to all connected clients that are subscribed to the relevant camera using a WebSocket connection. This occurs whenever a new detection or alert exists. Because of this, the dashboard can be updated in real time without the user having to do anything other than refresh the site.

In order to simplify the maintenance of WebSocket connections and provide valuable capabilities such as room-based broadcasting, which was used to deliver updates solely to users who were monitoring a particular camera, socket.io was selected as the platform of choice.

2.7 React and TypeScript

The application's frontend was constructed with the help of React [16], a JavaScript toolkit that is frequently utilized for the purpose of constructing user interfaces. Throughout the frontend, TypeScript was utilized to implement static typing, which not only assists in the detection of mistakes at an earlier stage but also makes the code simpler to maintain. A live monitoring dashboard, a page for controlling alarms, and an administrative panel for managing cameras are all included in the interface. In order to give a consistent visual style without creating each piece from scratch, Material UI [17] was utilized as the component library.

2.8 Redux Toolkit

Redux Toolkit [18] was used to handle the frontend application state. It includes the camera list, the latest detection data for each camera, the alert list and the presently authenticated user. We picked Redux because it is a situation where a centralized state management solution is working effectively, where several components need to share state and update it based on real time WebSocket events.

2.9 JSON Web Tokens and Authentication

The users were authenticated using JSON Web Tokens [19]. Whenever a user signs up, the server gives a signed token that contains the user's identity and his role in the system. With every request that follows this token is sent in order to verify the identity of the person making the request. The system was designed to handle three different roles: the normal user, the admin and the worker. Specifically, the worker role was for the Python detection service which authenticates its requests using a shared secret token that is stored in the environment settings of both services.

2.10 Kaggle

The Kaggle platform was used for training the YOLOv8 model [20]. Kaggle provides free access to graphics processing unit (GPU) computing power which significantly increased the training speed in comparison to what could be achieved on a regular personal computer. The training notebooks and dataset used for the purposes of this research were directly controlled within the platform.

2.11 Source control: Github

The whole development process was done using GitHub [21] for source control and version management. Git [22] was used to track the entire codebase including the frontend, backend and detecting worker. The codebase was kept in a private repository created on GitHub which gave a comprehensive history of the changes made during the development process and allowed for access to previous versions of the code when necessary.

2.12 Roboflow

During the period involved in the development of the dataset, Roboflow [23] was used. The platform provides tools that can be used to organize, classify and supplement picture collections for the purpose of performing object recognition tasks. In this project, Roboflow was used to create labels for the fire and smoke classes across all of the photos that were gathered and to export the final dataset in the format that YOLOv8 requires for training purposes. It also included augmentation options that supported the expansion of the training data's diversity without the need for manually collecting additional photos.

2.13 Conclusion

All the tools presented in this chapter had a specific purpose within the project. Some were in charge of data processing and detection, others dealt with storage and communication, and the others helped with the development process itself. The combination of these technologies allowed for the construction of a system that links all the necessary components, from reading a video frame to displaying a detection result in a live web interface.

Chapter 3: Model Training

The portion of the project that ended up taking the most time and being the one in which things went wrong the most frequently was teaching the model. It was more important to get this section right than the majority of the other decisions that were made during the development process because everything that the system does when it looks at a video frame is dependent on what the model learned before.

It begins with an explanation of why a custom model was required in the first place, then discusses the origin of the data, the challenges that arose throughout the process of cleaning and labeling it, and finally, it discusses the measures that were taken to address the gaps that were present in the dataset. At the very end, both the training event itself and the outcomes that it brought about are discussed.

3.1 Overview of the Approach

In the computer vision task of object detection, a model examines a picture to determine which items are there and where. In contrast to basic image classification, which just provides an answer to the query of what is contained in an image, object detection additionally creates a bounding box around each detected object and gives it a confidence score. This makes it appropriate for situations when several things can appear simultaneously in various areas of the frame, as is the case with smoke and fire. For this objective, there are numerous model families. Some analyze images slowly but with great precision because they value accuracy over speed. Others are made to operate as quickly as possible, which is particularly pertinent for real-time applications that require constant frame analysis.

The second category includes YOLO, which stands for You Only Look Once. The fundamental concept of YOLO is that the model looks at the entire image in a single pass and generates all of its predictions at once, as opposed to repeatedly scanning different areas of an image using a sliding window technique. Because of this, it is substantially quicker than previous detection techniques while retaining a level of accuracy suitable for practical application.

Ultralytics created YOLOv8, a sophisticated and thoroughly documented version. It featured a simpler Python interface that makes it simple to begin training without writing a lot of custom code, as well as a cleaner codebase and better default training configurations than previous versions like YOLOv5. Although more recent versions, such as YOLOv9, YOLOv10, and YOLOv11, have subsequently been released, YOLOv8 was selected because to its stability, the abundance of tutorials and community resources that are available, and the

quantity of useful information that has been developed around it. Changing to a newer, less tested version was not worth the additional uncertainty for a project that was more concerned with viability than performance. Since models built on standard datasets like COCO [24] do not include fire or smoke as detection classes, a pre-trained general purpose model was not used directly. A general model would not provide accurate bounding boxes for the particular appearance of fire and smoke in outdoor forest situations, even if it might tangentially link some visual patterns to these ideas. So to build a model trained on labeled examples of fire and smoke in a variety of real-world situations, we had to create a dataset and train from scratch using transfer learning on top of a pre-trained YOLOv8 basis.

3.2 Dataset collection

Locating appropriate training data was the first step. On sites like Roboflow and Kaggle, where some fire and smoke datasets had previously been uploaded by other academics and developers, a number of publicly accessible datasets were examined.

The issue was that each dataset was insufficiently complete on its own. Each one tends to address a limited number of situations. Some of the pictures were primarily taken at night, with the fire serving as the primary source of light, giving them a completely distinct appearance from footage taken during the day. Others concentrated on controlled burns or interior fires, which varied significantly in appearance from smoke and forest fires in open spaces. No one dataset contained the range of lighting conditions, distances, and backdrops required to train a model that functions consistently in many scenarios.

Given the above reasons, it was decided to merge multiple datasets into one. The notion was that combining the sources would at least increase the total variety of the training data, even if each source was restricted.

Following the selection of the datasets [25], [26], [27], and [28] that appeared to be the most pertinent and helpful, every image was collected into a single folder for examination and cleaning prior to labeling (see Figure 4). As a starting point, this method was the most practical way to work with what was publicly available, but it had its own set of issues, which are discussed in the parts that follow.



Figure 4: Some samples from the initial datasets

3.3 Dataset limitations

Even after merging several datasets, the final collection had several obvious limitations. Combining them only partially helped, and no single source was diversified enough on its own. Several recurring issues were identified while reviewing the photographs that are worth discussing here as they directly impacted the decisions made during cleaning, labeling, and augmentation (see Figure 5).

The main issues were:

- **Lighting:** in many of the photos, taken at night or in very low light, fire is the only bright element in the frame. This looks very different from how smoke and fire appear in the daytime, and a model mostly trained on these types of photos would likely struggle with images taken in daylight.
- **Lack of scene variety:** most of the photos were shot in the same environments, such as close-ups of flames or dense smoke. Images with fire partially obscured by trees or distant smoke on the horizon were either missing entirely or underrepresented.
- **Quality:** some of the pictures were fuzzy, highly compressed or from sources with very poor resolution making it difficult to draw precise bounding boxes during labeling.
- **Ambiguous cases:** many pictures showed burned areas without any apparent smoke or active fire, and some showed smoke that looked a lot like fog or dust clouds.
- **Noise in the dataset:** the examples introduced noise in the dataset and were hard to identify reliably.
- **Insufficient sample count for certain scenarios:** there were a fair number of pictures overall after merging, but very few samples for a number of specific scenarios, leading to the model not having much exposure to these scenarios during training.

These limitations could not be addressed completely but were considered when making decisions later on, especially during cleaning and augmentation.



Figure 5: Examples of images that were excluded during cleaning

3.4 Dataset cleaning

The first step was to de-duplicate all the photos from the various sources into a single folder. Many datasets combined together often mean the same image appears multiple times. To manage this automatically, a Python script was written comparing the hash of each image file. This removed a large number of exact duplicates without any manual inspection by deleting any image file that had a hash that had already been seen.

Next, each of the remaining images was visually inspected. This process involved manually going through the folder and removing anything that was obviously useless to the model. The criteria used were relatively simple, for example images that were too dark to see any detail, images whose content was unclear and could not be confidently labelled, images of very low quality as a result of compression or low resolution and images which, although part of a fire dataset, did not contain fire or smoke in any significant way. Some of the photos that showed aftermath scenes, such as burnt ground without smoke or active fire, were not relevant for a detection model designed to identify ongoing events.

After the deduplication process, roughly 7,800 photos from all the collected sources were part of the merged folder. At this stage, none of them had bounding boxes or class information associated with them, since all were unlabeled. The collection included images with only fire, only smoke, or both, as well as images with neither of these elements. The last category is important in the training to reduce the probability of false detections in the final model because the model needs to be exposed to normal natural scenarios to learn how to

distinguish them from real fire or smoke.

This was an important but time-consuming task. Adding irrelevant or low-quality photos to the dataset would have added noise to the training process and most likely reduced the practical reliability of the model.

3.5 Labeling

Most of the photos we collected had no annotations at all, and the few that did had such inconsistent annotations that it wasn't worth trying to reuse them. Bounding boxes varied in how tightly they were drawn, different sources used different definitions of classes, and some labels were just plain wrong. We decided to label the entire dataset from scratch, throwing out all existing annotations.

Roboflow was the platform for labeling. When the photographs were uploaded to the project, the pertinent objects were surrounded by bounding boxes and tagged with its label (see Figure 6). The two classes were fire and smoke .

The first half of the dataset was tagged fully by hand. It was a lot of work, but a solid base of annotations had to be produced before anything else could proceed. After labeling a sufficient number of photos, Roboflow used those labels to train a lightweight model that would suggest labels for the remaining images, greatly speeding up the process. The suggestions for each image were carefully reviewed, fixing boxes, adding missed detections, and deleting bad predictions as necessary. Every image in the final data set was checked at least once before being marked as complete.

The entire dataset with all annotations was exported using Roboflow in the YOLOv8-required format. A text file with the bounding box coordinates and class indices in normalized format is coupled with each image (see Figure 7). In order to inform the model that a certain image is a negative example with nothing to detect, the accompanying .txt label files for images without smoke or fire were left empty.

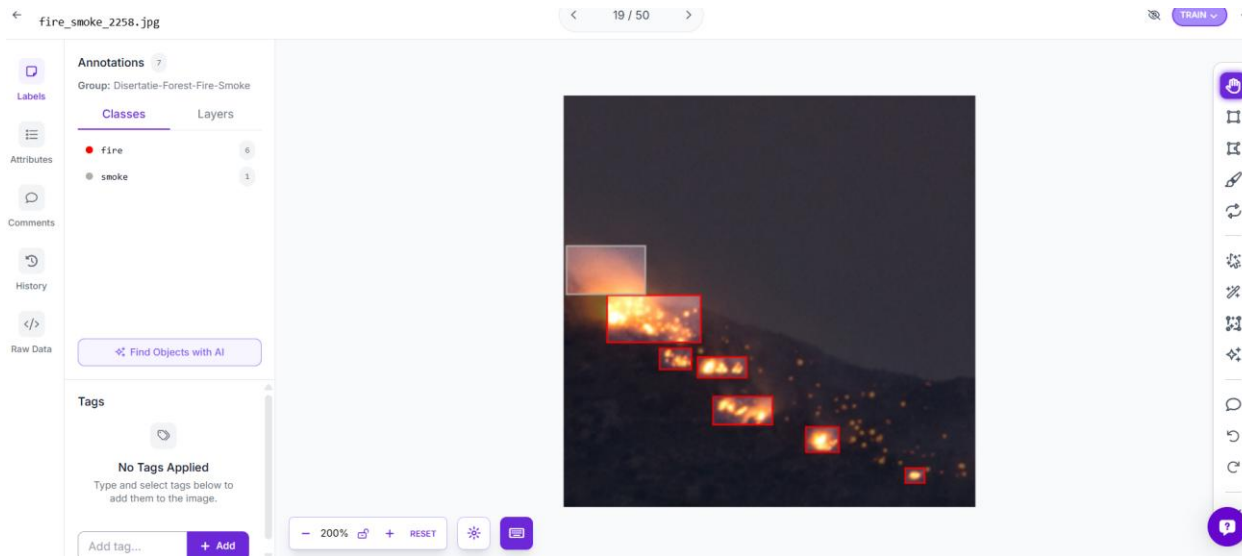


Figure 6: Example of a manually labeled image using Roboflow

```
File Edit View
0 0.819 0.81188 0.242 0.20292
0 0.6992 0.9542 0.1744 0.0916
1 0.509 0.37572000000000005 0.974 0.6466799999999999
1 0.5347999999999999 0.8412000000000001 0.2856 0.31760000000000005
```

Figure 7: YOLOv8 annotation showing bounding box coordinates and classes

3.6 Dataset augmentation

Augmentation was applied to two separate stages of the project, each with a different purpose.

Augmentation was first done inside Roboflow before the dataset was exported (see Figure 9). At this stage, simple changes such as horizontal flipping and adjustments to contrast and brightness were employed. The objective was to produce a slightly more diverse set of images for training the lightweight model which would aid in classifying the second half of the dataset, not to create the final training dataset. A simple augmentation setup was sufficient because the internal model only needed to be good enough to generate useful recommendations.

The final model was actually trained on Kaggle using the second and more important round of augmentation. YOLOv8 augments on the fly during training instead of creating new images on disk ahead of time. The changes are performed in memory every time an image is loaded for a training iteration, so the model sees a slightly modified version of each image

every epoch without the need for extra storage.

The augmentation settings configured for the training script were as follows (see Figure 8):

- **HSV adjustments:** random changes to brightness, saturation and hue to simulate different lighting situations and times of day
- **Horizontal flip:** 50% chance of applying a horizontal flip since smoke and fire have no fixed orientation
- **Scale and translation:** randomly resizing and moving the items allows the model to deal with objects at different distances and locations within the frame
- **Rotation and shear:** small geometric distortions to change the viewing angle
- **Mosaic:** four photos are combined into a single training sample with an 80% chance to force the model to identify items in heterogeneous situations and at smaller scales
- **Mixup:** a conservative method of blending two photos at a 5% chance to prevent excessive noise

These decisions were made intentionally to fill in the gaps seen in the dataset, especially the lack of variation in scene arrangement, lighting, and distance. Since gathering additional data was not a good option, augmentation was the primary method available to increase the diversity of what the model was exposed to during training.

```
results = model.train(  
    data="/kaggle/working/data.yaml",  
    epochs=60,  
    imgsz=640,  
    batch=16,  
    device=0,  
    hsv_h=0.01,      # hue adjustment  
    hsv_s=0.5,        # saturation adjustment  
    hsv_v=0.4,        # brightness adjustment  
    degrees=3,        # rotation  
    translate=0.08,    # translation  
    scale=0.4,         # scale  
    shear=1,          # shear  
    flip1r=0.5,        # horizontal flip (50% probability)  
    flipud=0.0,        # vertical flip (disabled)  
    mosaic=0.8,        # mosaic (80% probability)  
    mixup=0.05,        # mixup (5% probability)  
    workers=2,  
    cache=True,  
    name="forest_fire_detector"  
)
```

Figure 8: Augmentation configurations used for training

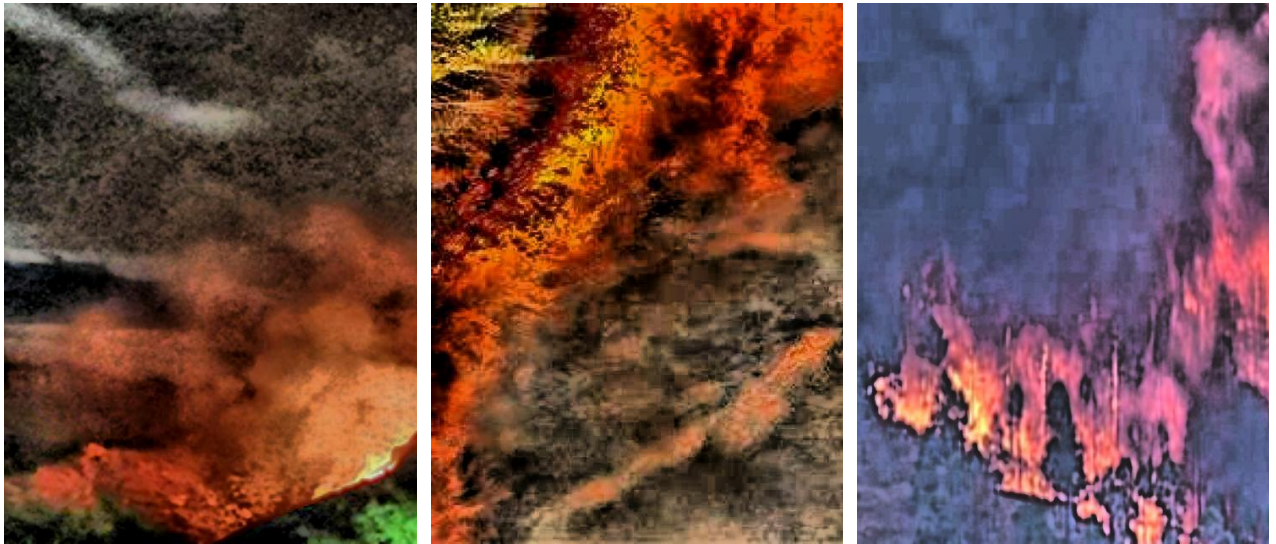


Figure 9: Augmented images used for the partial Roboflow model

3.7 Training process

Training of the model was done on Kaggle using Tesla T4 GPU offered on the free tier environment of the platform. The 16GB VRAM on the Tesla T4 was sufficient for the size of the dataset and the batch configuration. The ability to train was made possible because Kaggle offered free access to GPU resources thereby eliminating the need for specialized hardware.

Loading of the base model was a tiny version of YOLOv8, YOLOv8s.pt, pretrained on the COCO dataset. Transfer learning is a well-known technique where one starts with pretrained weights instead of training from scratch. The model has already learned general visual features such as edges, shapes and textures from its initial training. Training from scratch takes more time and data than fine-tuning it on the fire and smoke dataset.

The training was configured with the following parameters:

Parameter	Value
Epochs	80
Image size	640 x 640
Batch size	16
Optimizer	AdamW
Learning rate	0.001667

Based on the model size and dataset size, YOLOv8 was trained using AdamW [29] with a learning rate of 0.001667, momentum 0.9, and an auto setting for the optimizer. The validation set contained 1,101 photos and 576 images with no detections, while the training set contained 5,144 images, including 2,690 images with no detections. Several corrupt files were found and automatically skipped during loading.

One interesting feature of YOLOv8 is the automatic disabling of mosaic augmentation in the last ten epochs. While this is helpful during training, Mosaic creates an artificial distribution that is different from true single-image inference by combining four images into a single training sample. Before the end of the training the model can be turned off on the last epochs to stabilize and refine on more realistic inputs. This can be seen in the loss curves, where when mosaic is turned off the class loss drops significantly at epoch 70 (see Figure 10).

The entire training run took about two hours to complete. YOLOv8.pt automatically saved the best checkpoint as **best.pt** and the last checkpoint as **last.pt** having the highest mAP50 score achieved on the validation set across all epochs.

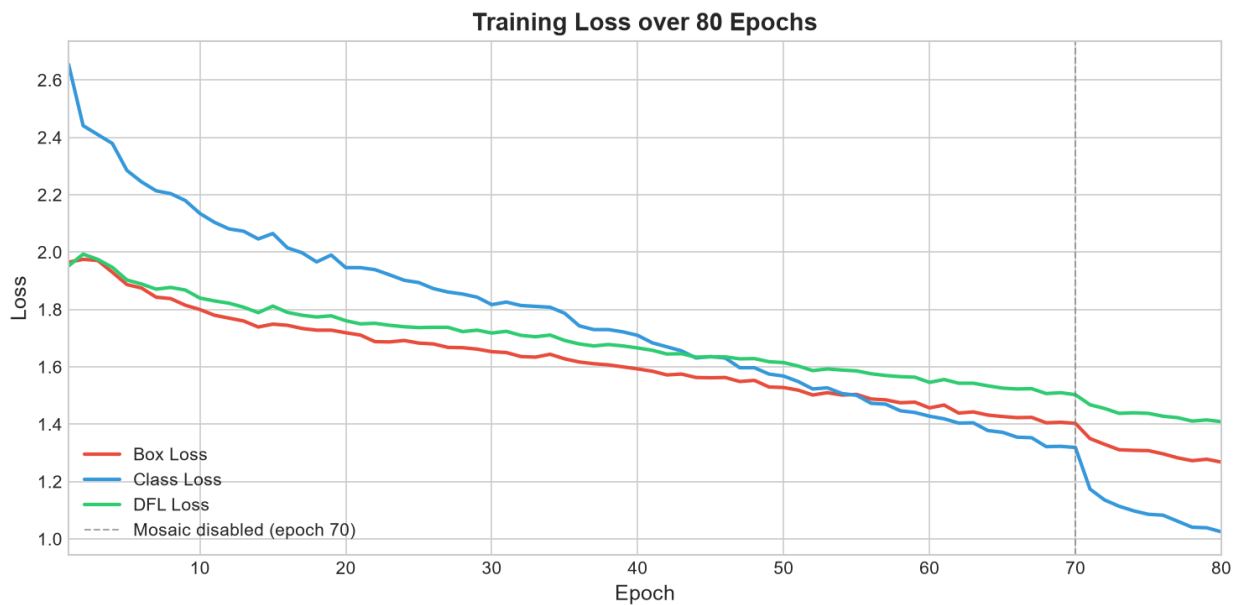


Figure 10: Training loss for 80 epochs

The YOLOv8 model was trained using the AdamW optimizer because of the model and dataset configurations. The AdamW optimizer is an improved version of the Adam optimizer and has weight decay. Weight decay is a regularization technique to prevent the model from overfitting to certain patterns in the training data and improve the model's ability

to generalize to new images . It is one of the most reliable and commonly used optimizers for training neural networks .

3.8 Results and evaluation

The best checkpoint was saved at epoch 54, where the model reached its highest validation mAP50 of 0.455. The full evaluation of this checkpoint on the validation set produced the following results:

Class	Precision	Recall	mAP@50	mAP@50-95
Fire	0.525	0.572	0.544	0.249
Smoke	0.465	0.398	0.366	0.153
Overall	0.495	0.485	0.455	0.201

It is important to provide a concise explanation of what each metric represents before moving on to the discussion of the numbers.

The term "precision" refers to the degree to which a model is able to avoid false alarms by determining the percentage of the observed detections that were actually accurate. Recall is a measurement that determines how many of the actual instances of fire or smoke in the images were successfully detected. This indicates how few actual cases the model fails to identify.

In the field of object detection, the most important summary metric is known as mAP@50. This metric represents the average precision across all classes at an IoU threshold of 50%. More specifically, it indicates that a detection is considered accurate if the predicted bounding box overlaps the ground truth by a minimum of fifty percent. This version is more stringent than the previous one, as it takes the average of the metric across multiple thresholds ranging from fifty percent to ninety-five percent. It rewards not only detection but also accurate localization of the object.

The evolution of the validation metrics over the course of the 80 training epochs is depicted in Figure 11. The mAP50 began at approximately 0.19 and continued to see steady improvement throughout the first half of the training session, reaching approximately 0.45 by the middle of the session before beginning to slow down and level off. The fact that the model had absorbed the majority of the information that the dataset had to teach it at that point is evidenced by the fact that this progression is normal. Considering the dataset and the inconsistencies that it contains, it is not surprising that the precision and recall curves

fluctuate more than usual.

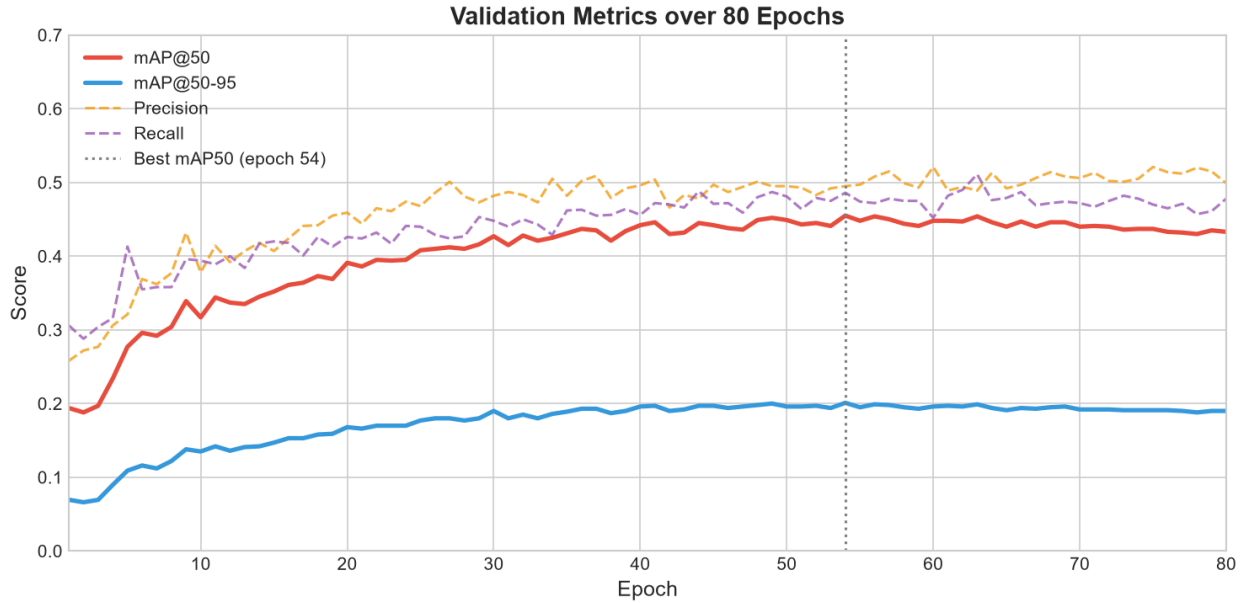


Figure 11: Validation metrics on 80 epochs - mAP@50, mAP@50-95, precision, and recall

The final performance is broken down by class in Figure 12, which can be found [here](#). Fire detection performed significantly better than smoke detection, with a mAP50 of 0.544 as opposed to 0.366 for smoke detection. This difference is not surprising. Fire has a consistent visual appearance, with bright colors and strong contrast with the background. This makes it easier for the model to learn reliable detection patterns because fire has a consistent appearance against the background.

Smoke however can have a high degree of variation in terms of its density, color and shape. In certain circumstances, smoke can have a similar appearance to fog or haze. During the process of labeling, these ambiguities were already identified, and it is evident that they are reflected in the lower scores for this particular class.



Figure 12: Final model performance on the validation set

On paper, a mAP50 value of 0.455 may appear to be low; however, this value does not accurately reflect how the model performs in actual simulations. Although the detections are visually correct, the score on the validation set is penalized by the fact that the labels were manually drawn and can not be considered perfectly consistent. When testing the model on real video footage from the internet, presenting forest fires, active smoke and normal outdoor scenes, the model correctly detected fire and smoke in most of the cases and produced very few false detections. This means that the practical performance of the model is better than the metric itself.

3.9 Conclusion

It turned out that building the detection model was more of a data problem than a training problem. The model itself was easy to set up and train, but the quality and diversity of data set were the main factors that determined the final result. A mAP50 of 0.455 is a reasonable result for a first attempt with a manually labeled custom dataset and practical testing on real footage confirmed that the model works well enough to serve its purpose in this project. The main takeaway is that improving the dataset would have a greater impact on the performance than any change to the training configuration.

Chapter 4: System Architecture

All of the information regarding the detection model and how it was trained was presented in the prior chapter. Within the scope of this chapter, the manner in which that model was incorporated into a more comprehensive system as well as the overall appearance of that system are discussed.

It was not enough to simply have a model that is capable of detecting smoke and fire in order to construct a monitoring platform that was functional. Having a web interface that allows users to view what is happening in real time, a server that can receive and process the results, a database that can store relevant data, and a way to continuously feed video input into the model were all necessary components. For the system to function as a whole, each of these components had to be designed and connected in a manner that satisfied the requirements of the system.

It is essential to keep in mind that this system is a testing concept before delving into the specifics of each individual component to be discussed. Neither real camera hardware nor drones nor live video streams are utilized in this process. The detection worker, on the other hand, receives video files as input and processes them in the same manner as if they were streams of live video. In order to validate the architecture, this approach was selected because it enables the entire pipeline to be developed, tested, and demonstrated without the need for physical infrastructure. Furthermore, it simulates the behavior of a real monitoring system in a manner that is close enough to the real thing.

The structure of the system, the decisions that were made over the course of its design, and the reasoning that led to those decisions are all described in the following sections.

4.1 System Overview

There are three primary components that make up the system. These components operate independently of one another and communicate with one another using HTTP and WebSockets. Specifically, the Python detection worker, the NestJS backend server, and the React frontend application are the components concerned. Each component fulfills a specific function, and no component is responsible for the work that is performed by another component (refer to Figures 13 and 14).

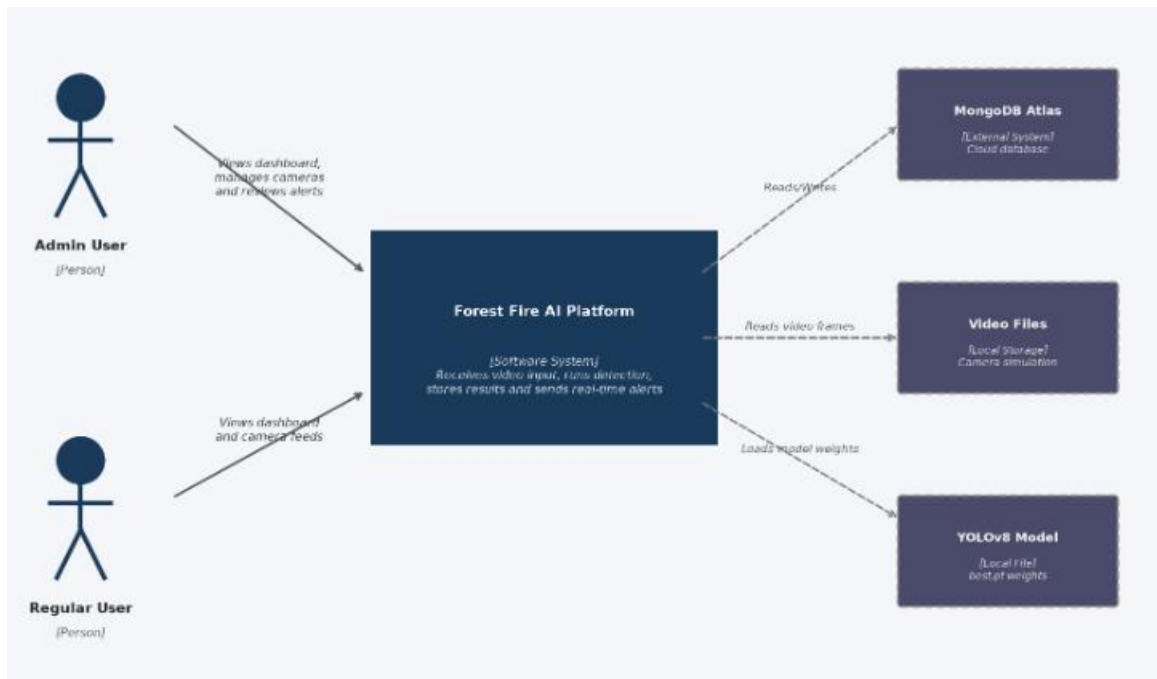


Figure 13: C1 Context Diagram of the system

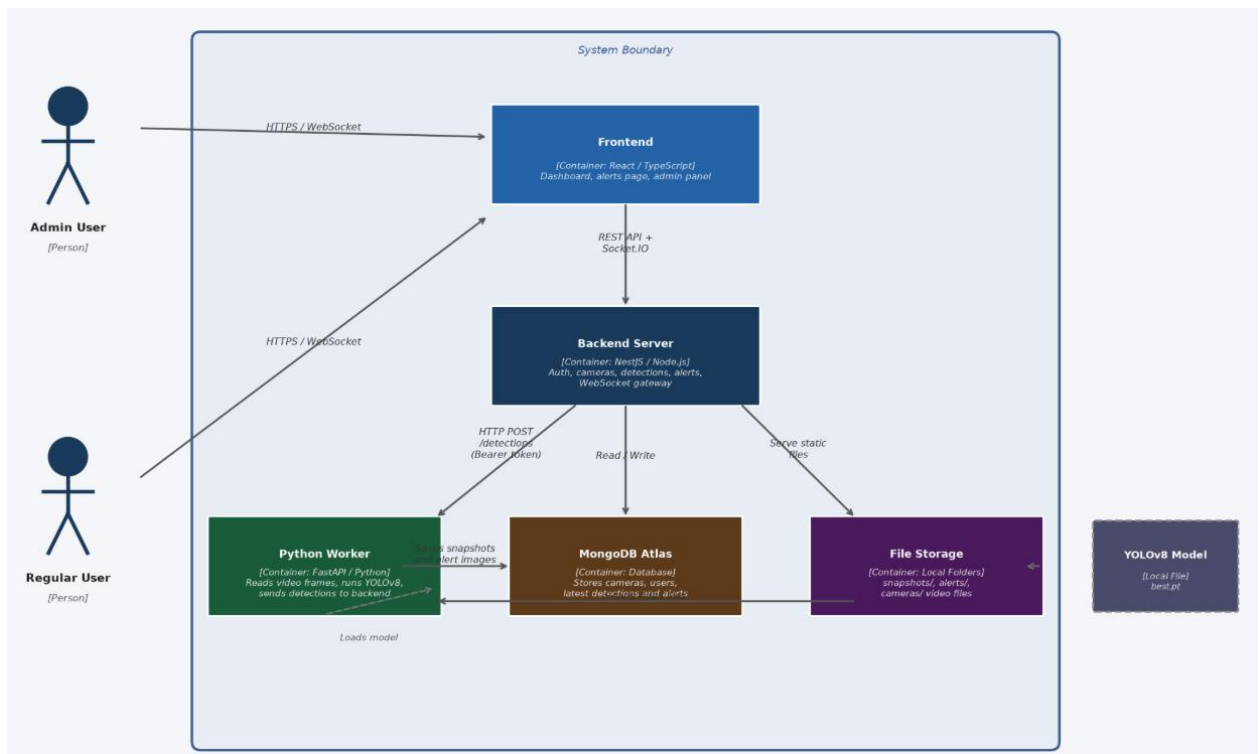


Figure 14: C2 Container Diagram of the system

Before elaborating on the manner in which the components interact with one another, it is essential to make it clear what kind of system this is. There is no use of live video streams, drones, or physical cameras in this process. The video files that are kept in a local folder that is referred to as cameras/ are instead provided to the worker who is responsible for

detection. The processing of each file is carried out as if it were a continuous live feed, and each file represents a camera source. The decision was made to take this approach in order to maintain the system's full testability without necessitating the use of any physical hardware, while simultaneously covering the entire pipeline, beginning with video input and ending with real-time alerts in the browser (see Figure 15).



Figure 15: Full System Architecture Diagram

The data flow through the system is as follows:

- Administrator logs in to the platform and adds the cameras using the admin panel. This includes providing the name of the camera, its location and the filename of the video file that corresponds to the selected camera and is stored in the cameras/ folder
- The Python worker calls the backend to get all the cameras marked as active. It also generates a separate processing thread for each of the cameras
- Each thread opens the video file that has been assigned to it and starts reading frames at the time interval that has been configured. This allows the threads to behave as if a live continuous feed
- The YOLOv8 model takes each frame as input and generates a list of detections with class label and confidence score. It also provides an annotated version of the frame with the bounding boxes drawn on it
- The annotated frame is stored as a JPEG to the snapshots/ folder, always overwriting the same file for that camera. This ensures that only the most recent image is stored on the disk.

- The worker is responsible for transmitting the detection result to the backend by means of an authenticated HTTP POST request. The worker sends the detection result to the backend via an authenticated HTTP POST request, carrying a shared bearer token to prove its identity
- The token is verified by the backend, the camera document in MongoDB is updated with the latest detection data, and a risk level is computed using the confidence scores
- A new WebSocket event is broadcast to all frontend clients subscribed to the room of the camera, which triggers the immediate update of the camera card on the dashboard. In the meantime, the backend is determining whether the detection qualifies as an alert.
- If the fire or smoke confidence is greater than seventy percent and at least five minutes have passed since the last alert for that camera, a copy of the snapshot is saved to the alerts/ folder, and an alert:new event is pushed to all connected admin clients in real time

The platform is designed to accommodate two distinct categories of users, each of which has a distinct level of access (see Figure 16):

- An individual who is a regular user has the ability to log in, view the live monitoring dashboard, and interact with individual camera reports. A detail dialog with a live video tab that plays the footage from the most recent analyzed timestamp can be opened, and they are able to view the most recent detection snapshot for each camera, as well as the objects that have been detected and the confidence levels associated with them.
- An administrator has access to everything that a regular user can see, in addition to the ability to manage cameras through the admin panel. Through this panel, an administrator can add, edit, delete, and activate or deactivate individual camera sources. Administrators also have access to the alerts page, which displays all of the alerts that have been generated along with snapshots, risk levels, and detection details relevant to each alert. It is possible to review alerts and either acknowledge them or mark them as resolved. When new alerts arrive in real time, they are sent directly to admin clients that are connected to the system through the WebSocket connection. This makes it unnecessary to refresh the page.

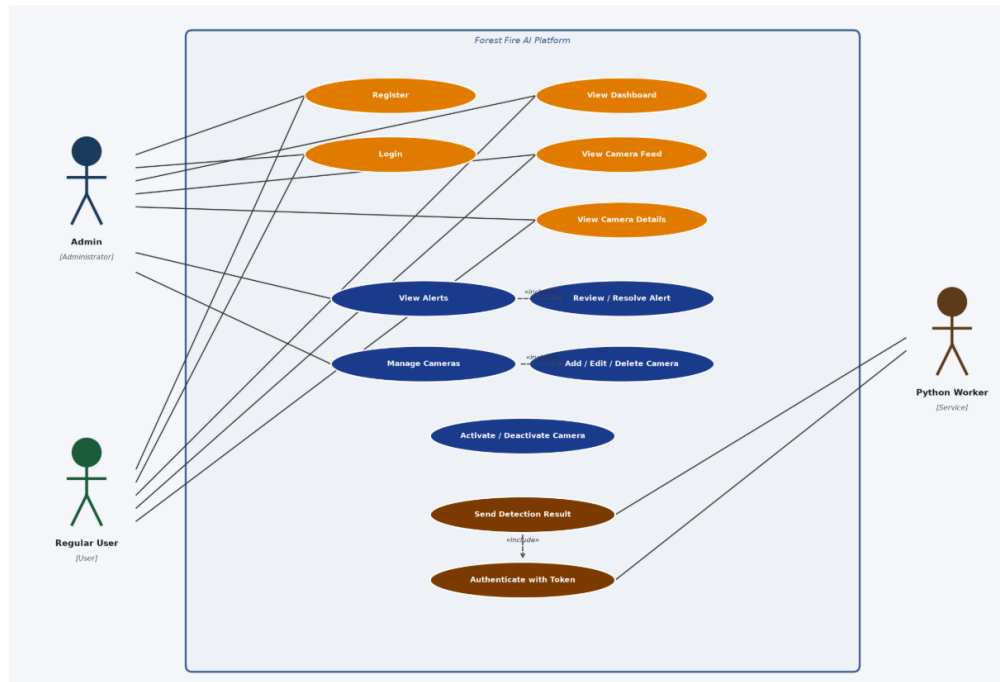


Figure 16: Use Case Diagram of the System

4.2 Database Structure

The system uses MongoDB as its database which is hosted remotely on MongoDB Atlas. Unlike relational databases where data is kept in a fixed set of rows and columns, MongoDB [12] is a document based database where data is stored in flexible JSON-like objects. The data in this system is simple enough that a relational database would have had needless overhead with a lot of tables and joins to represent structures that easily map to a single document. MongoDB allowed for storing the camera configuration and its most recent detection in one place, which allows for easier code to work with and straightforward queries.

The database is comprised of three collections: users, cameras, and alerts (see Figure 17). The users collection holds the roles and credentials of all people who have access to the platform. Each document contains the user's name, email address, hashed password, and role. The role field can be one of three values: regular user, administrator, and worker. The last role is used to authenticate the backend HTTP requests of the Python detection service.

The most important part of the system is the collection of cameras. Each document contains the name, location, region, source filename, and analysis settings of a monitored camera source. Also, it has an embedded latestDetection object that is updated whenever the employee sends a new detection for that particular camera. This embedded strategy was a deliberate design choice. Because the frontend only needs to display the latest detection for each camera, tracking every detection would lead to an ever-growing database without any added benefits to the user. The queries are fast and the documents are small when each camera only has the most recent detection. The latestDetection object has the time of the

detection, the URL of the snapshot saved, the video time in milliseconds, the calculated risk level, and an array of detection items. Each detection item stores the class name, confidence score, and bounding box coordinates of a single detected object within the frame. The alerts collection keeps records of noteworthy detection events that exceeded the alert threshold.

Alerts are permanently stored because, unlike detections, they are intended to be examined and responded to. Each alert document refers to the camera from which it originated by the camera's ID, and includes the detection timestamp, detection type, maximum confidence score, risk level, URL of the saved alert snapshot image, and a status field that indicates whether the alert has been reviewed.

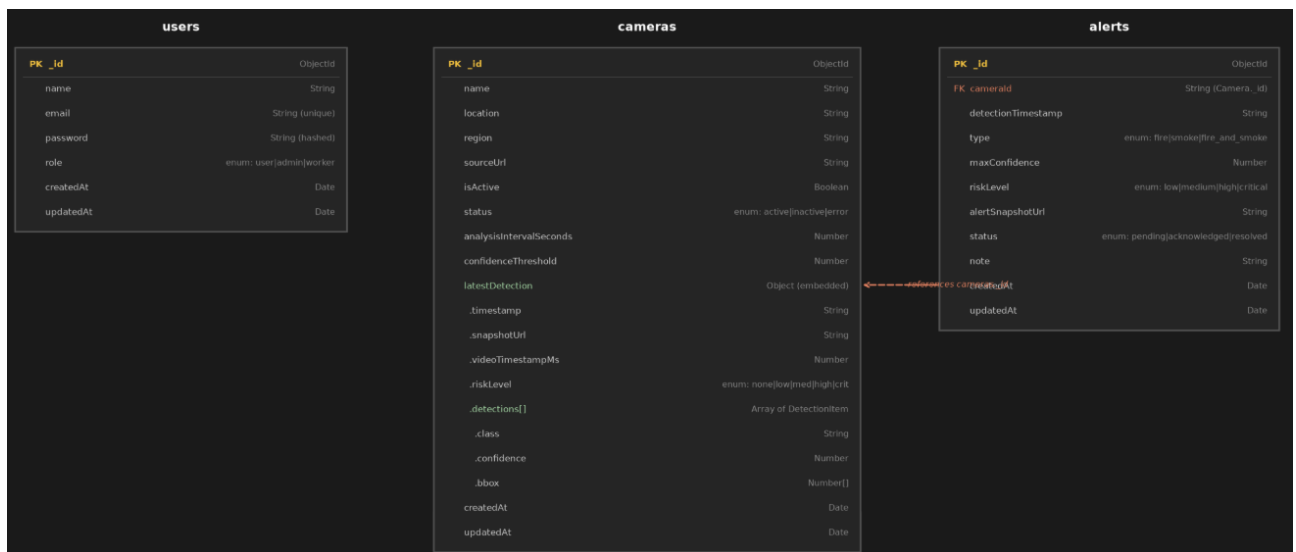


Figure 17: Database Schema

4.3 Python Detection Worker

This component is responsible for reading video input, running the detection model, and reporting results to the backend. It is a completely independent service, communicating with the backend only via HTTP.

Service structure

The worker is built on FastAPI and provides a small set of endpoints that allow the backend to query the status of all active workers and to start and stop individual camera threads. When the worker starts, it makes a call to the backend health endpoint to verify it is accessible. If the backend is unreachable, the worker will exit immediately instead of starting in a broken state. If the check is successful, a thread for processing is spawned for each live camera, after fetching the list of cameras from the backend and filtering out the inactive ones. To reduce the overhead of loading the YOLOv8 model more than once, it is loaded once at

startup and shared amongst all threads.

Threading model

Each camera is handled by a central WorkerManager class that runs in its own thread. Each thread has its own unique stop signal and video capture handle. When a stop command is received for a given camera, the signal is set and the thread completes its current processing cycle before exiting cleanly. Because they are set up as daemon threads, they automatically terminate when the main process does.

Frame processing

Each thread jumps straight to a target frame position and reads just that one frame of the video instead of reading each frame in turn. Once processed, the target position is pushed forward by the number of frames corresponding to the configured analysis interval. The video loops indefinitely, simulating a continuous feed, and the position is reset to the start when the end of the video is reached. The thread sleeps the length of the analysis interval in between cycles, but it does so in a way that allows a stop signal to wake it up immediately, rather than waiting for the whole interval to end.

Snapshot saving

Using that camera's same filename, the model's annotated frame is saved as a JPEG to the snapshots folder. Only the most recent annotated frame is ever stored on disk as the file is overwritten on each cycle. There is no collection of historical snapshots.

Delivering outcomes to the backend

The JSON payload that the worker sends to the backend after saving the snapshot includes the camera ID, a timestamp, the video position in milliseconds at the time of the analyzed frame, the snapshot URL, and a list of detected objects with their class, confidence score, and bounding box coordinates. The Authorization header of each request contains a bearer token, which the backend uses to confirm that the request originates from the authorized worker service. If the backend can't be reached, if the request times out after five seconds, or if it gets an unexpected response, the error is logged and the thread continues on without delay to the next cycle.

4.4 Backend Server

The backend (NestJS) is the heart of the system. It handles the database, processes all

incoming requests from the frontend and the python worker, and uses WebSockets to implement real-time communication with connected clients.

Authentication and roles

User authentication is implemented by using JSON Web Tokens [19]. The backend verifies the user's login information and returns a signed token. A JWT guard checks the token sent with every next request before the request reaches any protected route. The system has three roles: worker, administrator and regular user. Route access is controlled by a roles guard that checks the role embedded in the token. The worker role is specifically used by the Python detection service to authenticate using a shared bearer token that is checked by a dedicated guard by comparing the value of the token with the value stored in the server environment configuration.

Camera management

The cameras module makes available a complete set of REST endpoints for the creation, reading, updating and deletion of camera records. The backend calls the start or stop endpoint of the Python worker when a camera is activated or deactivated via the admin panel in order to keep the processing state in sync with the database state.

Detection processing

Once the worker sends a detection to the backend, the detection service first derives a risk level based on the confidence scores of the payload. It then updates the embedded latestDetection field in MongoDB for the camera, and all frontend clients subscribed to the room of that camera receive an immediate detection:new WebSocket event. To determine whether the detection is an alert, it then invokes the alert service.

Alert threshold logic

Before it creates an alert, the alert service checks two things. The first of which is whether the maximum fire confidence or maximum smoke confidence for the detection is at least 70%. The check ends there if neither requirement is satisfied.

Second, it determines whether the last alert was generated for that camera at least five minutes ago. A map that keeps track of the most recent alert timestamp for each camera ID is used to monitor this cooldown in memory. The service creates a new alert document in MongoDB with a status of pending, copies the current snapshot from the snapshots folder to a dedicated subfolder inside the alerts folder named after the camera ID, and sends an alert:new WebSocket event to all clients in the admins room if both conditions are met. The alert type is

then determined based on which classes triggered the threshold.

WebSocket gateway

Socket.IO is the foundation of the WebSocket gateway and oversees two different kinds of rooms. Only clients who are currently viewing a particular camera can receive detection:new events from camera rooms, which are named after each camera ID. Admin clients connect to a separate room called the admins room, which is only used to send alerts about new events. As they navigate the dashboard and alter which cameras are visible, clients dynamically subscribe to and unsubscribe from camera rooms.

Static file serving

The snapshots, alerts, and cameras folders are served by the backend as static directories, so the frontend doesn't need any extra file handling logic to load annotated snapshot and alert images by URL.

Why the backend and the worker are separate services

Maintaining the NestJS backend and the Python detection worker as separate processes instead of combining them into a single application is one choice that merits explanation.

Performance is the most pragmatic explanation. It takes a lot of computing power to run a machine learning model. Every frame analysis would compete with incoming HTTP and WebSocket requests for resources if the inference logic operated within the same process as the web server. Regular users' response times would significantly deteriorate during active detection times, particularly if several cameras are operating simultaneously.

The second justification is stability. If the detection worker crashes due to a memory problem, an unhandled exception in the model pipeline or a corrupted video file, the backend remains operational. Users can still access the dashboard, log in and see previous alerts. One component problem does not bring the whole system down.

The third justification is separation of concerns. The backend handles real-time communication, business logic, data persistence and authentication, while the worker does model inference and video processing. By merging the two into one service, it becomes more difficult to maintain and scale the code base independently. If detection needs more processing power down the line, the worker can be moved to a more powerful machine without changing the backend.

Last but not least, keeping the services separate allows for more flexibility in the choice of technology. The backend is implemented in NestJS and TypeScript because they

work well with Websockets and REST APIs. The worker is implemented in Python as Python is the base of the machine learning ecosystem including Ultralytics, OpenCV and numpy. If the two were merged one side would have to make an uncomfortable compromise

4.5 Frontend Application

We used Material UI as the component library and Vite as the build tool, the frontend is a single-page application created with React and TypeScript. Redux Toolkit manages all global state, while React Router handles page navigation [30]

Routing and application structure

A landing page, login page, registration page, dashboard, alerts page, and admin cameras page are among the pages that make up the application. The user must be logged in in order to access the dashboard and alerts page. The admin role is also needed for the admin cameras page. A user is redirected appropriately if they attempt to access a page that they are not permitted to view.

Management of the states

The cameras slice and the alerts slice are the two primary slices of state that are maintained by Redux Toolkit [18]. A separate map of each camera's most recent detection, indexed by camera ID, and the complete list of cameras retrieved from the backend are both stored in the cameras slice. By keeping detections in a separate map, only the components that read the data from that particular camera re-render and only the pertinent entry in that map is updated when a new detection arrives over WebSocket. The list of alerts retrieved during page load is stored in the alerts slice, which also gathers newly received alerts in real time.

WebSocket integration

One socket is used when a user logs in. For the duration of the session, an IO connection is established and maintained. This connection is shared between components via a context provider and is controlled by a custom hook named useSocket.

An internal list of currently subscribed camera IDs is kept up to date by the hook. The hook sends subscribe or unsubscribe messages to the backend in accordance with the camera IDs that are currently displayed on the screen when the dashboard renders visible camera cards. This puts the client in a camera-specific socket on the backend. IO rooms. Detection:new event instantly updates the redux store and the corresponding camera card updates the snapshot and detection results without a page refresh or polling. The hook also re-emits all the currently selected subscriptions in case the socket connection drops and

reconnects preventing any silent camera feed. The admin users also get alerts about new events by subscribing to the admins room on connection. In the event such occurrence does happen, the new alert is added to the redux alerts list

Dashboard page

All the cameras are fetched from the backend and added to the redux store when the dashboard loads. If a camera already has a latestDetection stored in the DB, that data is loaded immediately to avoid the dashboard looking empty on the initial load. At the top of the page are two summary cards, one showing the total number of cameras with the active and inactive counts separated, and the other showing unreviewed alerts separated by risk level. Only admin users can see the second card. Below the summary cards is the camera grid, which shows the most recent annotated snapshot and risk level for each camera card. Clicking on a camera card brings up a detail dialog with two tabs, one for the detection view and one for live video playback

Alerts page

The alerts page is only accessible to admin users. On mount, it fetches all the alerts from the backend and displays them as cards in a paginated grid of nine alerts per page. Each card displays the alert snapshot image, camera name, detected class, confidence score, risk level, timestamp and current status. A dropdown menu can be used to filter the cards by status, allowing the user to display all alerts, only pending alerts, only acknowledged alerts or only resolved alerts. Each card has action buttons for acknowledging or resolving the alert.

These buttons trigger the backend and change the status badge of the card without reloading the page, updating the Redux store immediately. When a new alert comes in via WebSocket, a short notification appears at the bottom of the screen when the user is on this page. Clicking on a snapshot image will open a fullscreen lightbox dialog.

Admin cameras page

Administrators can manage the list of cameras on the admin cameras page. It provides a table where cameras can be added, modified and removed. When creating or editing a camera, the administrator fills in the name of the camera, stream source path and if it is active or not in the form dialog that appears. When changing the active state of a camera through the form, a call to the backend is made to tell the python worker to start or stop processing the camera stream.

4.6 Conclusion

Every component of the system serves a single purpose. The worker runs the model and reads the video. The backend determines when to generate alerts and stores data. The frontend displays current events and updates as they occur.

It made sense to keep the worker and the backend apart because running a model inside the web server would have made it slow under load. Because there is no need to store every frame result, the database schema was intentionally kept simple, retaining only the most recent detection per camera. By sending each client only what it is actually looking at, WebSocket rooms maintain a clean traffic stream.

Chapter 5: Application Presentation

In this chapter the application is broken down into individual pages to see what has been built and what it looks like when it is being used. Each section deals with a different part of the user interface and describes what the user can see and do in that part.

5.1 Landing Page

The landing page (see Figure 18) is the first thing a visitor sees when they open the application. It provides a brief overview of the platform, with a title and description, before the user logs in. There are two buttons: one to make a new account and one to log in with an existing one. The design is deliberately simple as the page has no functional use other than to direct the user to authentication.

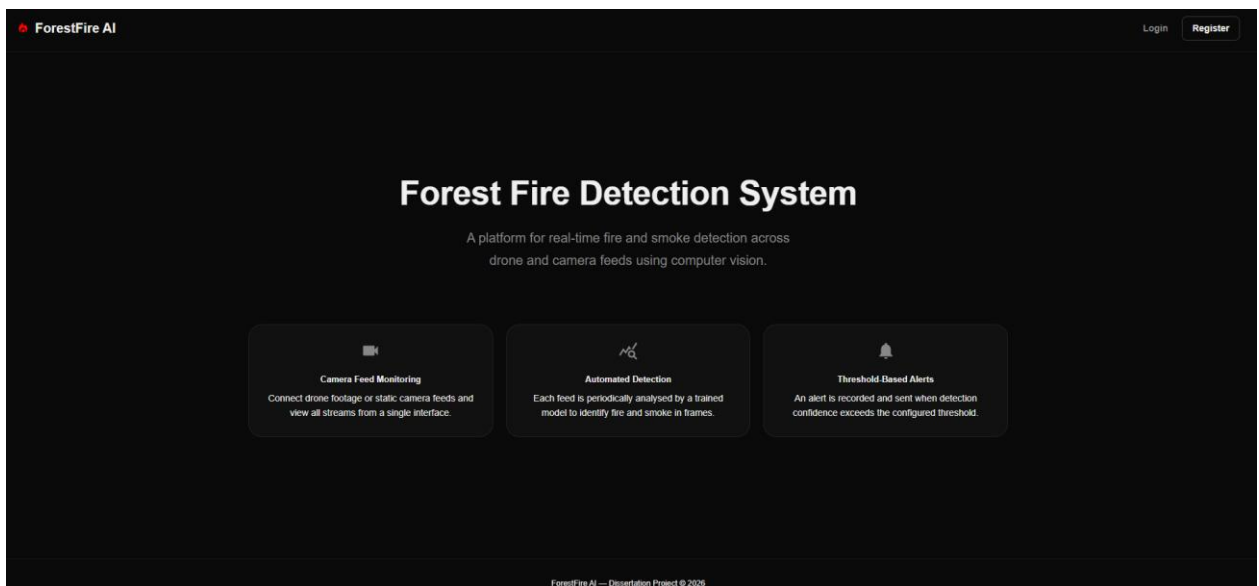
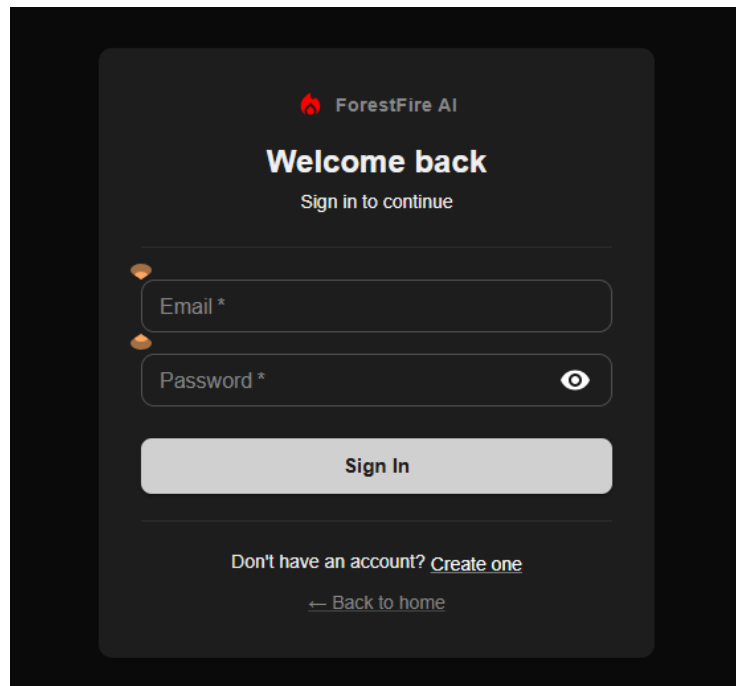


Figure 18: Landing Page

5.2 Authentication

On the login page, the user is presented with a straightforward form that requires them to enter their e-mail address and password (see Figure 19). Before the request is sent, the form is validated to ensure that it is correct. This ensures that any obvious errors, such as a field that is empty, are caught immediately. If the user is able to successfully log in, they will be taken to the dashboard, where they will receive a JWT token from the backend. This token will be saved in the browser. An error message will be displayed below the form in the event that the credentials do not correspond to an existing account.

The image shows a login page for ForestFire AI. At the top, there is a logo with a red flame icon and the text "ForestFire AI". Below the logo, the heading "Welcome back" is displayed in a bold, white font, followed by the subtext "Sign in to continue" in a smaller, lighter font. The login form consists of two input fields: "Email *" and "Password *". The "Password *" field has a toggle icon (an eye) to its right. Below the input fields is a large, light gray button labeled "Sign In". At the bottom of the form, there is a link "Don't have an account? Create one" and a link "← Back to home".

ForestFire AI

Welcome back

Sign in to continue

Email *

Password * 

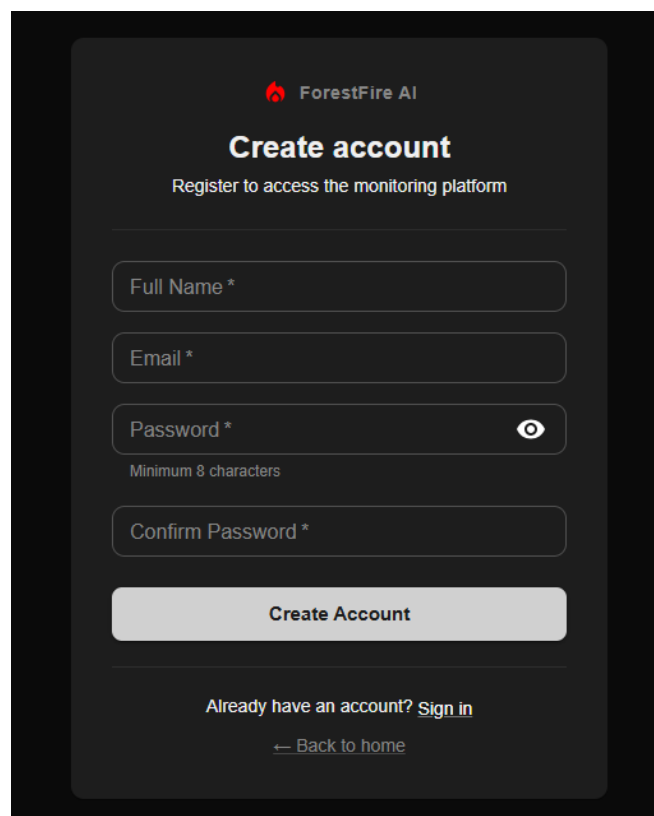
Sign In

Don't have an account? [Create one](#)

[← Back to home](#)

Figure 19: Login Page

You will be required to provide a display name, an email address, and a password on the registration page (see Figure 20), which follows the same structure. There is no need for the user to log in separately because once the account has been created, they are automatically authenticated and taken to the dashboard.

The image shows a registration page for ForestFire AI. At the top, there is a logo with a red flame icon and the text "ForestFire AI". Below the logo, the heading "Create account" is displayed in a bold, white font, followed by the subtext "Register to access the monitoring platform" in a smaller, lighter font. The registration form consists of four input fields: "Full Name *", "Email *", "Password *", and "Confirm Password *". The "Password *" field has a toggle icon (an eye) to its right. Below the "Password *" field, there is a text label "Minimum 8 characters". Below the input fields is a large, light gray button labeled "Create Account". At the bottom of the form, there is a link "Already have an account? Sign in" and a link "← Back to home".

ForestFire AI

Create account

Register to access the monitoring platform

Full Name *

Email *

Password * 

Minimum 8 characters

Confirm Password *

Create Account

Already have an account? [Sign in](#)

[← Back to home](#)

Figure 20: Register Page

5.3 Dashboard

Once logged in, both types of users are directed to the dashboard, which is the main page of the application (see Figure 21). The top contains two summary cards. The first shows the total number of cameras in the system and the number of active cameras. The second shows the number of unreviewed alerts broken down by risk level using color-coded labels. Under the summary section is the camera grid. Each camera is represented by a card showing the camera name, the most recent annotated snapshot from the detection worker and a badge showing the current risk level. The grid updates in real time as new detections come in over the WebSocket connection, so the snapshots and risk badges update automatically without needing to be refreshed manually.

The second can only be seen by administrators and shows the number of unreviewed alerts broken down by risk level using color-coded labels. Under the summary section is the camera grid. Each camera is represented by a card showing the camera name, the most recent annotated snapshot from the detection worker and a badge showing the current risk level. The grid updates in real time as new detections come in over the WebSocket connection, so the snapshots and risk badges update automatically without needing to be refreshed manually.

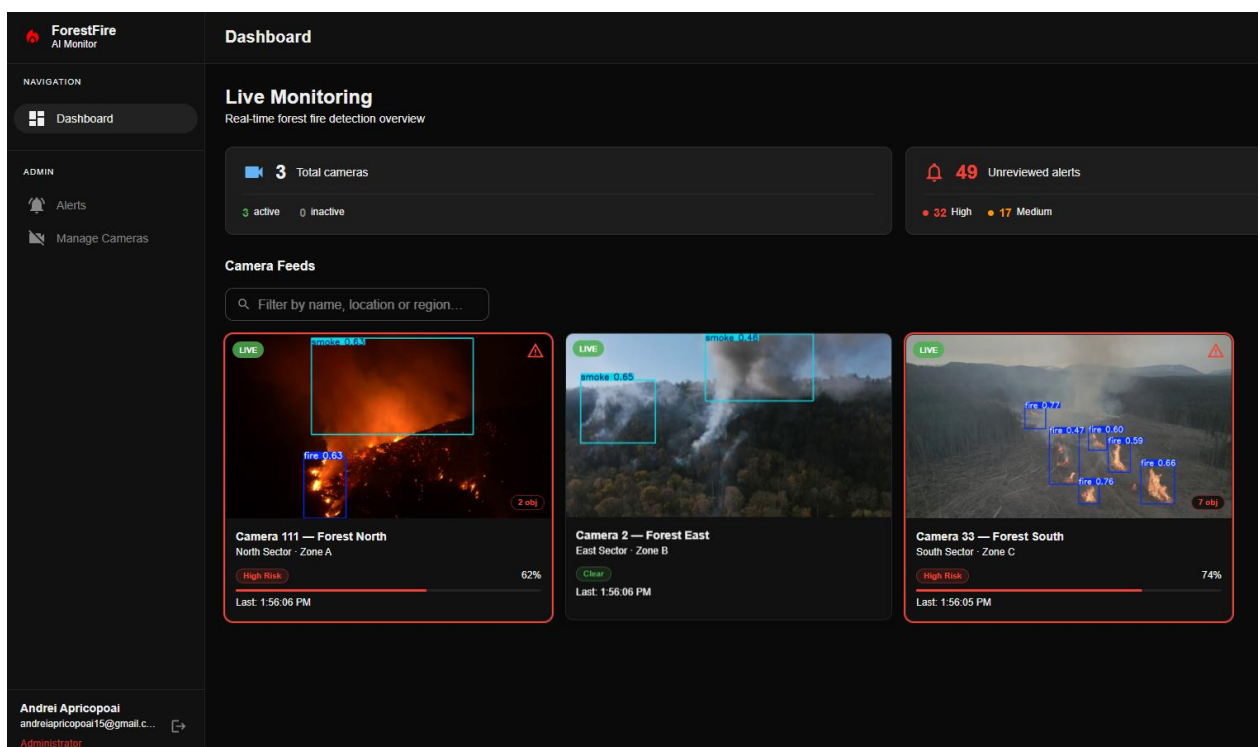


Figure 21: Dashboard Overview

5.4 Camera Detail Dialog

A detail dialog (see Figure 22) appears at the top of the dashboard when you click on any camera card. There are two tabs in the dialog. The first tab is the detection view, showing the latest snapshot image in an enlarged view with information on the detection such as the class found, confidence score, and risk level assigned, for easier viewing of the detected objects. The video player (see Figure 23) is the second tab. It loads the stream source set up for that camera and allows the user to watch the video in the browser without ever leaving the page.

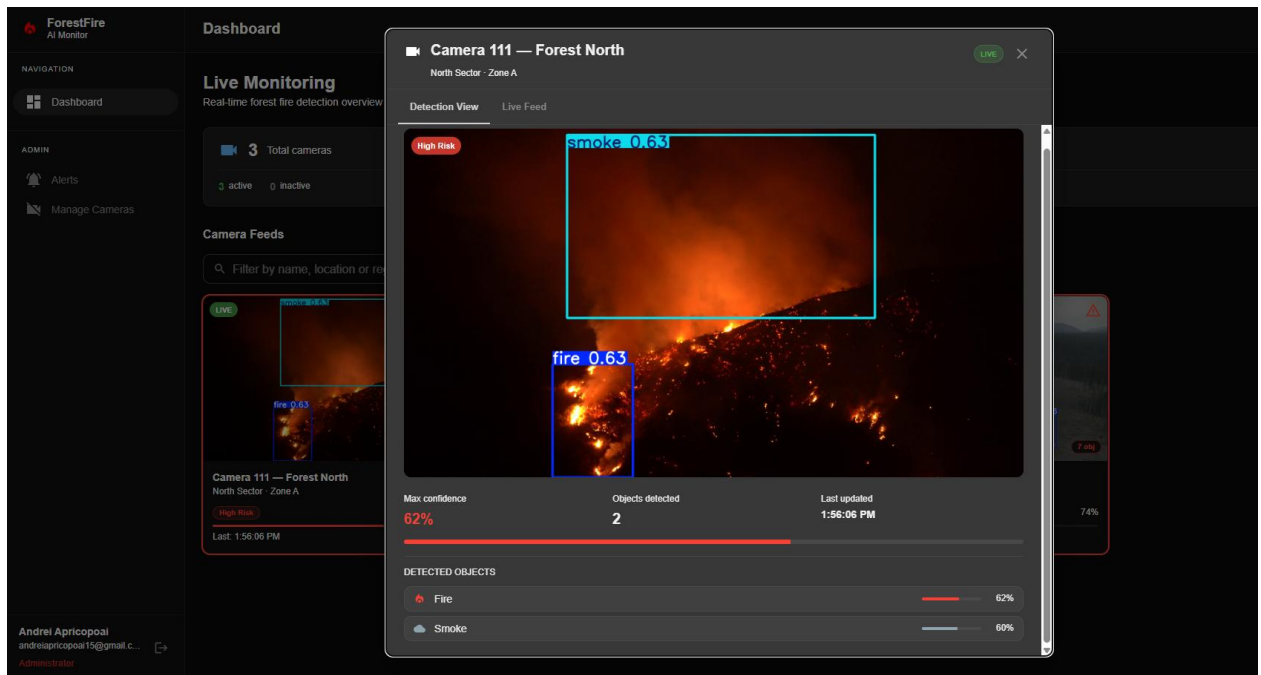


Figure 22: Camera detail dialog - detection view tab

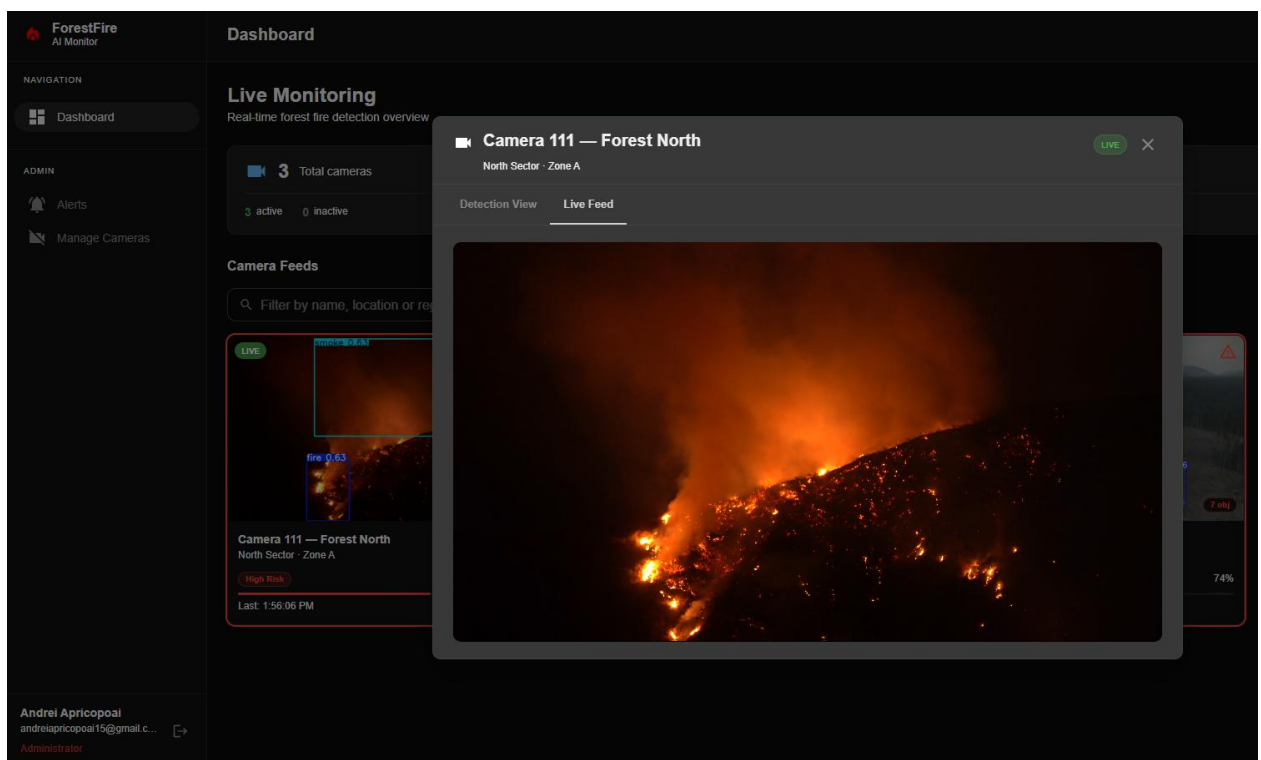


Figure 23: Camera detail dialog - live video tab

5.5 Alerts Page

Only administrators can access the alerts page (see Figure 24), which displays the complete history of alerts produced by the system. A pagination control at the bottom allows users to navigate through older alerts, which are presented as a grid of nine cards per page. Each card provides details such as the name of the camera that made the detection, detection

type (fire, smoke or both), confidence level, risk level, timestamp, snapshot image at the moment the alert was triggered, and current status of the alert. The administrator has the ability to filter alerts by status through a dropdown menu in the top right corner with four options: all, pending, acknowledged, and resolved. Action buttons are provided to acknowledge or mark the alerts as unresolved. Both actions apply immediately and update the status on-screen without a page reload. Clicking on the snapshot image of any of the alert cards will open it in a fullscreen (see Figure 25).

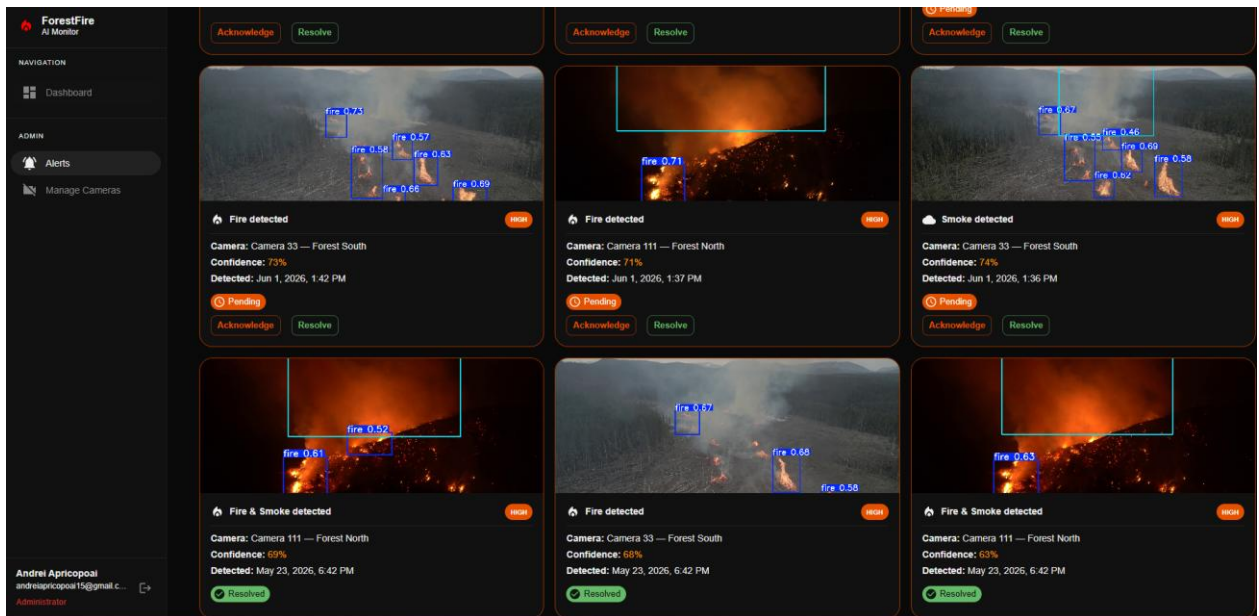


Figure 24: Alerts page with alert cards

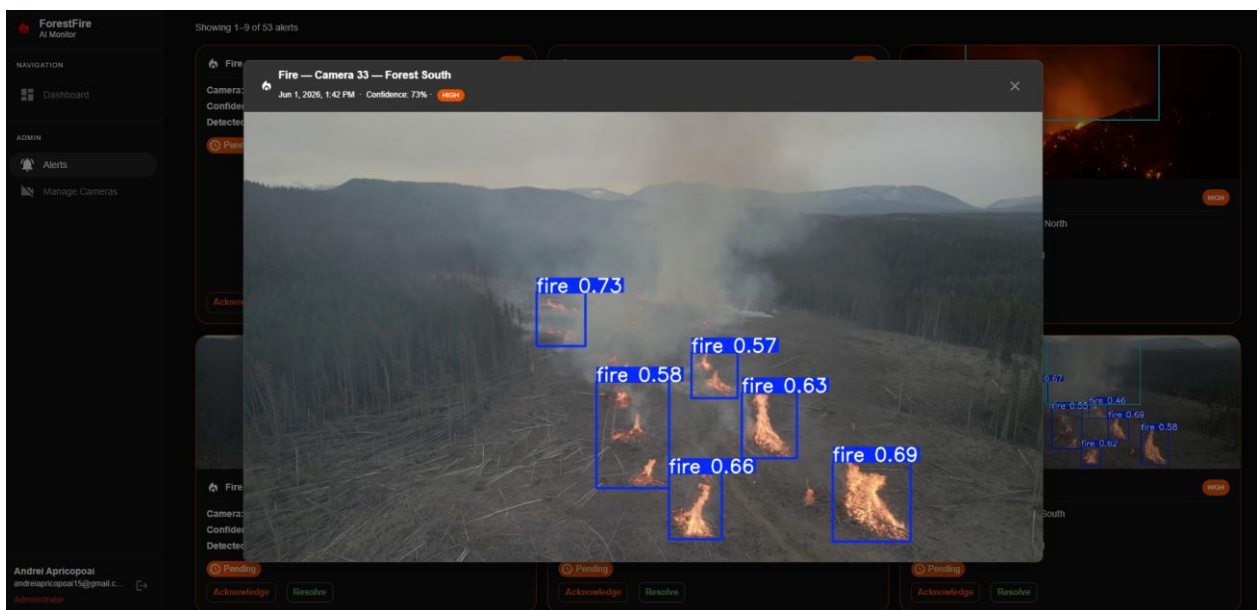
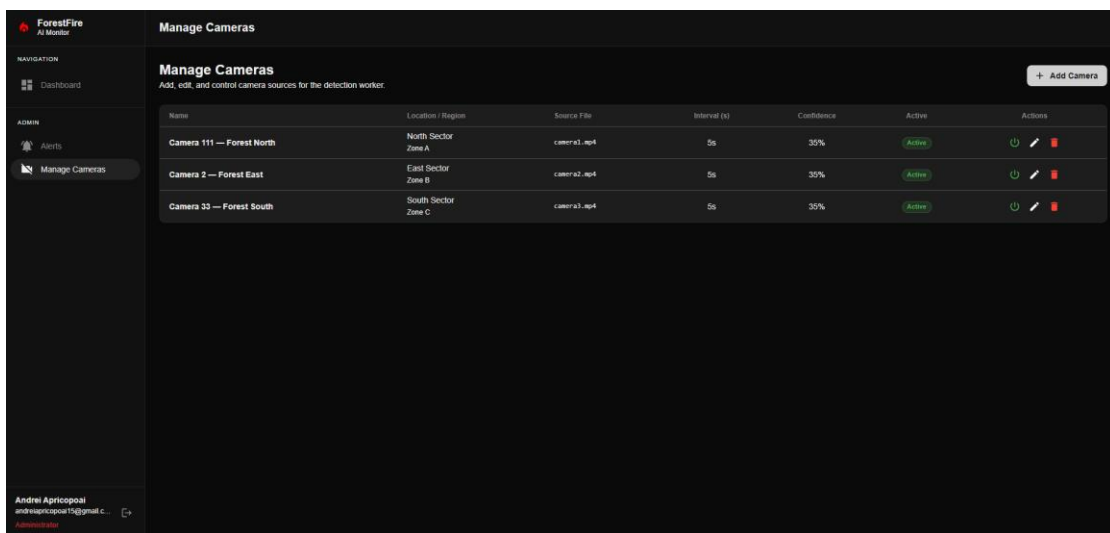


Figure 25: Alert image opened in fullscreen

5.6 Admin Cameras Page

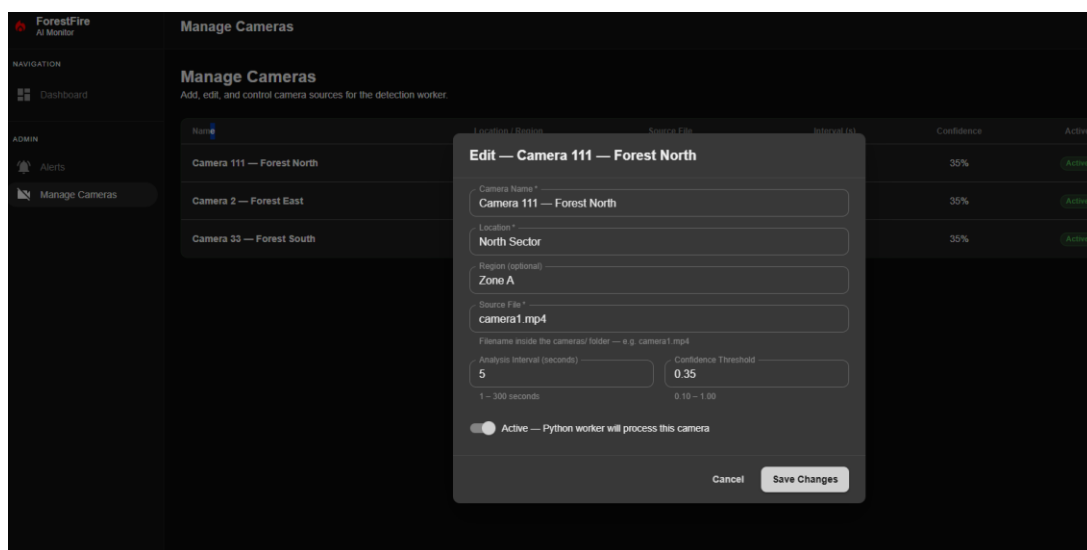
With the admin cameras page the administrators can register cameras in the system and have full control over it. There is a table (see Figure 26) that shows all the cameras with the file path or stream source used for video input, the camera name and whether it is active or not. An administrator can use the button at the top of this page to add a new camera (see Figure 27), edit an existing camera by clicking the edit icon on its row, or remove it completely by using the delete button. A modal dialog with fields for the name, the stream source path, and a toggle for the active state appears when a camera is added or edited.

If you save a camera with the toggle on, the backend will tell the Python worker to start a new processing thread. If you turn it off, the thread will be stopped and detection for that stream will stop.



Name	Location / Region	Source File	Interval (s)	Confidence	Active	Actions
Camera 111 — Forest North	North Sector Zone A	camera1.mp4	5s	35%	Active	
Camera 2 — Forest East	East Sector Zone B	camera2.mp4	5s	35%	Active	
Camera 33 — Forest South	South Sector Zone C	camera3.mp4	5s	35%	Active	

Figure 26: Admin cameras table



Edit — Camera 111 — Forest North

Camera Name *
Camera 111 — Forest North

Location *
North Sector

Region (optional)
Zone A

Source File *
camera1.mp4
Filename inside the camera/ folder — e.g. camera1.mp4

Analysis Interval (seconds) Confidence Threshold
1 – 300 seconds 0.10 – 1.00

☒ Active — Python worker will process this camera

Cancel Save Changes

Figure 27: Add or edit camera dialog

5.7 Conclusion

The application has all the information to fully illustrate the idea of a forest fire detection system. It allows administrators to see alerts, control their lifecycle and which cameras are being monitored by the system, and it allows users to see the camera feeds as detections happen in real time.

The way the interface behaves is a reflection of the design choices made during development, so it does not require a manual refresh and the page responds to updates coming through the WebSocket connection.

An administrator can go from seeing a new alert to resolving it in just a few clicks. Managing alerts is easy. Adding or removing streams is easy and does not require any configuration files to be changed thanks to the camera management panel. The prototype interface effectively addresses the main use case and would provide a strong foundation for future system development.

Final Conclusions and Possible Improvements

Conclusions

The aim of this project was to develop a working prototype of a forest fire monitoring system using advanced web technologies and a custom-trained machine learning model. This goal was achieved in terms of what was delivered. The detection worker uses a YOLOv8 model trained on a custom dataset to process video streams frame-by-frame, run inference and immediately send results to the backend. The backend uses WebSockets to send real-time updates to connected clients, store detection data, apply alert logic and deal with authentication. The frontend is used to display everything in a clear interface and automatically update when new detections arrive. The whole process, from looking at a video frame to the alert showing up in a browser, is working and all three parts work together as a system.

The model works well enough for a research prototype. The validation metrics were quite modest, due to the limitations of the training dataset, but the model reliably detects smoke and fire in real-world video footage. One big lesson learned from this project is the difference between the numbers and the real behavior, which shows the importance of the quality of the dataset in object detection tasks.

There are flaws in the current implementation. There is no built-in fault tolerance, the alert cooldown resets if the server restarts, and video files are used instead of live camera feeds. These were design choices to keep the project in a realistic scope of a research prototype, not bugs.

Possible Improvements

The biggest change would be switching the static video files for actual camera hardware or live RTSP streams. This would bring the system from a simulation to something that would actually be useful in a real monitoring situation.

It is possible to greatly increase and purify the training dataset. The ability of the model to generalize would improve with more diverse images covering different lighting conditions, weather, vegetation types and fire stages. Increasing the labeling consistency throughout the dataset would also bring the validation metrics closer to the real-world performance of the model.

The alert cooldown could be moved from in-memory storage to the database. If the backend restarts, the cooldown resets and the next qualifying detection will immediately trigger a new alert, regardless of when the previous alert was saved. This could be solved by storing the most recent alert timestamp for each camera in MongoDB. Real operators would find the interface more useful if it included user-configurable notification preferences, email or push notifications for new alerts, and a historical view of detection data per camera. Lastly, the deployment could be strengthened. The system would be closer to something that could be left unattended for long periods of time if the worker and backend were run as managed services with automatic restarts, appropriate logging, and health monitoring.

Use of AI Tools

AI tools were used in the following ways during the preparation of this paper:

- Improving the grammar and phrasing of all sections, based on drafts written by the author
- Supporting the architectural design process as a thinking and reasoning aid, helping to validate decisions and explore trade-offs
- Generating Figures 13, 14, and 15, which represent the system architecture diagrams
- Identifying possible future improvements to the system discussed in the Final Conclusions and Possible Improvements section
- Suggesting a baseline configuration for the initial training script used on Kaggle, including parameters such as epoch count, batch size, and optimizer selection

Bibliography

- [1] “Forest Fires in Europe, Middle East and North Africa.” [Online]. Available: <https://forest-fire.emergency.copernicus.eu/>
- [2] “Wildland Fire Statistics,” National Interagency Fire Center. [Online]. Available: <https://www.nifc.gov/fire-information/statistics>
- [3] “Pano AI Wildfire Detection,” Pano AI. [Online]. Available: <https://www.pano.ai/>
- [4] “Silvanet Wildfire Detection,” Dryad Networks. [Online]. Available: <https://www.dryad.net/>
- [5] “Welcome to Python.org,” Python.org. [Online]. Available: <https://www.python.org>
- [6] “FastAPI Documentation,” FastAPI. [Online]. Available: <https://fastapi.tiangolo.com>
- [7] “Ultralytics YOLOv8 Documentation,” Ultralytics. [Online]. Available: <https://docs.ultralytics.com>
- [8] R. Joseph, S. Divvala, G. Ross, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” presented at the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.
- [9] “OpenCV - Open Source Computer Vision Library,” OpenCV. [Online]. Available: <https://opencv.org>
- [10] “NestJS - A Progressive Node.js Framework,” NestJS. [Online]. Available: <https://docs.nestjs.com>
- [11] “Node.js Documentation,” Node.js. [Online]. Available: <https://nodejs.org>
- [12] “MongoDB Documentation,” MongoDB. [Online]. Available: <https://www.mongodb.com/docs>
- [13] “Mongoose - Elegant MongoDB Object Modeling for Node.js,” Mongoose. [Online]. Available: <https://mongoosejs.com>
- [14] “MongoDB Atlas - Cloud Database Service,” MongoDB. [Online]. Available: <https://www.mongodb.com/atlas>
- [15] “Socket.IO Documentation,” Socket.IO. [Online]. Available: <https://socket.io/docs>
- [16] “React - The Library for Web and Native User Interfaces,” React. [Online]. Available: <https://react.dev>
- [17] “MUI - The React Component Library,” MUI. [Online]. Available: <https://mui.com>
- [18] “Redux Toolkit Documentation,” Redux. [Online]. Available: <https://redux-toolkit.js.org>
- [19] “JSON Web Token (JWT) - RFC 7519,” Internet Engineering Task Force. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [20] “Kaggle - Your Machine Learning and Data Science Community,” Kaggle. [Online]. Available: <https://www.kaggle.com>
- [21] “GitHub Documentation,” GitHub. [Online]. Available: <https://docs.github.com>
- [22] “Git - Documentation,” Git. [Online]. Available: <https://git-scm.com/doc>
- [23] “Roboflow - Computer Vision Developer Platform,” Roboflow. [Online]. Available: <https://roboflow.com>
- [24] “COCO - Common Objects in Context,” COCO Dataset. [Online]. Available: <https://cocodataset.org>
- [25] datascientist97, “Forest Fire Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/datascientist97/forest-fire>
- [26] kutaykutlu, “Forest Fire Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/kutaykutlu/forest-fire>

- [27] “Forest Fire Dataset - alma,” Roboflow Universe. [Online]. Available: <https://universe.roboflow.com/alma-qkjpw/forest-fire-lh4m9>
- [28] “Forest Fire Dataset - object-detection,” Roboflow Universe. [Online]. Available: <https://universe.roboflow.com/object-detection-n6mxu/forest-fire-tyoo7>
- [29] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations (ICLR)*, 2019, [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [30] “React Router Documentation,” React Router. [Online]. Available: <https://reactrouter.com>